A PROJECT REPORT ON

AI INTRUSION DETECTION SYSTEM USING MACHINE LEARNING AND SPLUNK ENTERPRISE

A Network Traffic Classification and SIEM Visualization Platform Based on the Random Forest Algorithm and the NSL-KDD Dataset

Prepared in IEEE Reference Format

Submitted by

Bhargav Naidu. S
B.Tech — Cyber Security and Digital Forensics
VIT Bhopal University

05 August 2026

ABSTRACT

The rapid growth of network-dependent digital infrastructure has increased the volume and sophistication of cyber attacks, exposing the limitations of traditional signature-based Intrusion Detection Systems (IDS) in identifying previously unseen threats. This report presents an AI Intrusion Detection System that applies a Random Forest Classifier, trained on the benchmark NSL-KDD dataset, to classify network traffic as Normal or Attack. A Flask-based web application allows users to upload network traffic in CSV format and view classification results through an interactive dashboard featuring summary statistics, confidence scores, charts, and downloadable reports. Prediction outcomes are converted into structured security logs and indexed into Splunk Enterprise, a widely adopted Security Information and Event Management (SIEM) platform, enabling real-time monitoring and visualization of intrusion events. The resulting platform unifies machine learning, web development, and SIEM technologies into a single, modular, and extensible system, achieving high classification accuracy while providing an accessible interface for security analysis. Experimental evaluation on the NSL-KDD dataset demonstrates the effectiveness of the proposed approach, and the report concludes with a discussion of the system's advantages, limitations, and directions for future work, including real-time packet capture and deep-learning-based classification.

Index Terms — Intrusion Detection System, Machine Learning, Random Forest, NSL-KDD, Flask, Splunk Enterprise, SIEM, Network Security.

TABLE OF CONTENTS

ABSTRACT 2
TABLE OF CONTENTS 3
CHAPTER 1 9
INTRODUCTION 9
1.1 Introduction 9
1.2 Background 9
Signature-Based Detection 10
Anomaly-Based Detection 10
1.3 Intrusion Detection System (IDS) 10
1.4 Machine Learning in Intrusion Detection 10
1.5 Splunk Enterprise Integration 11
1.6 Motivation 11
1.7 Scope of the Project 11
Chapter Summary 11
CHAPTER 2 12
PROBLEM STATEMENT 12
2.1 Introduction 12
2.2 Existing System 12
Characteristics of Existing Systems 12
2.3 Problems in Existing Systems 12
1. Inability to Detect Unknown Attacks 12
2. Frequent Signature Updates 12
3. High False Positive Rate 13
4. High False Negative Rate 13
5. Manual Analysis 13
6. Lack of Intelligent Decision Making 13
2.4 Need for the Proposed System 13
2.5 Proposed System 13
2.6 Proposed System Architecture Overview 13
2.7 Advantages of the Proposed System 14
Intelligent Detection 14
Higher Accuracy 14
User-Friendly Interface 14
Interactive Visualization 14
SIEM Integration 14

Automated Reporting 14
Security Log Generation 14
2.8 Applications 14
Chapter Summary 14
CHAPTER 3 16
OBJECTIVES 16
3.1 Introduction 16
3.2 General Objective 16
3.3 Specific Objectives 16
Objective 1 – Develop an AI-Based Intrusion Detection Model 16
Objective 2 – Utilize the NSL-KDD Dataset 16
Objective 3 – Perform Data Preprocessing 16
Objective 4 – Develop a Flask Web Application 16
Objective 5 – Generate Prediction Reports 16
Objective 6 – Display Interactive Visualizations 16
Objective 7 – Generate Security Logs 16
Objective 8 – Integrate Splunk Enterprise 17
Objective 9 – Improve Detection Accuracy 17
Objective 10 – Build a Scalable Security Solution 17
3.4 Expected Outcomes 17
Chapter Summary 17
CHAPTER 4 18
LITERATURE SURVEY 18
4.1 Introduction 18
4.2 Intrusion Detection Systems 18
Host-Based Intrusion Detection System (HIDS) 18
Network-Based Intrusion Detection System (NIDS) 18
4.3 Signature-Based Detection 18
4.4 Anomaly-Based Detection 18
4.5 Machine Learning in Intrusion Detection 19
4.6 Random Forest Classifier 19
4.7 NSL-KDD Dataset 19
4.8 Security Information and Event Management (SIEM) 19
4.9 Research Gap 19
Chapter Summary 20
CHAPTER 5 21

SYSTEM REQUIREMENTS 21
5.1 Introduction 21
5.2 Hardware Requirements 21
Processor 21
Memory 21
Storage 21
5.3 Software Requirements 21
5.4 Python Libraries 22
5.5 Development Environment 22
Chapter Summary 22
CHAPTER 6 23
TECHNOLOGIES USED 23
6.1 Introduction 23
6.2 Python 23
6.3 Flask 23
6.4 Pandas 23
6.5 NumPy 23
6.6 Scikit-learn 23
6.7 Bootstrap 23
6.8 HTML 23
6.9 CSS 23
6.10 JavaScript 24
6.11 Chart.js 24
6.12 Git & GitHub 24
6.13 Splunk Enterprise 24
Chapter Summary 24
CHAPTER 7 25
DATASET 25
7.1 Introduction 25
7.2 NSL-KDD Dataset 25
7.3 Dataset Files 25
7.4 Dataset Features 25
7.5 Data Preprocessing 26
Data Cleaning 26
Feature Selection 26
Label Encoding 26

Data Transformation 26
7.6 Dataset Workflow 26
7.7 Advantages of NSL-KDD 26
7.8 Limitations of the Dataset 26
Chapter Summary 26
CHAPTER 8 28
SYSTEM ARCHITECTURE 28
8.1 Introduction 28
8.2 System Architecture Diagram 28
8.3 Architecture Components 28
8.3.1 User Interface 28
8.3.2 CSV Upload Module 28
8.3.3 Data Preprocessing Module 28
8.3.4 Machine Learning Prediction Module 28
8.3.5 Prediction Dashboard 28
8.3.6 Report Generation Module 29
8.3.7 Security Log Generation 29
8.3.8 Splunk Enterprise 29
8.4 Overall Workflow 29
8.5 Advantages of Modular Architecture 29
Chapter Summary 29
CHAPTER 9 30
METHODOLOGY 30
9.1 Introduction 30
9.2 Phase 1 – Dataset Collection 30
9.3 Phase 2 – Data Preprocessing 30
9.4 Phase 3 – Model Training 30
9.5 Phase 4 – Web Application Development 30
9.6 Phase 5 – Prediction 30
9.7 Phase 6 – Dashboard Generation 30
9.8 Phase 7 – Report Generation 30
9.9 Phase 8 – Log Generation 30
9.10 Phase 9 – Splunk Integration 30
9.11 Methodology Flow 31
Chapter Summary 31
CHAPTER 10 32

RANDOM FOREST ALGORITHM 32
10.1 Introduction 32
10.2 Working Principle 32
10.3 Random Forest Workflow 32
10.4 Why Random Forest? 32
10.5 Advantages 32
10.6 Disadvantages 32
10.7 Why It Was Used in This Project 33
10.8 Performance 33
Chapter Summary 33
CHAPTER 11 34
SYSTEM IMPLEMENTATION 34
11.1 Introduction 34
11.2 Project Directory Structure 34
11.3 Module Description 34
11.3.1 Dataset Module 34
11.3.2 Model Module 34
11.3.3 Preprocessing Module 34
11.3.4 Prediction Module 34
11.3.5 Flask Application 34
11.3.6 Dashboard Module 34
11.3.7 Log Generation Module 35
11.3.8 Splunk Integration Module 35
11.4 Execution Flow 35
Chapter Summary 35
CHAPTER 12 36
USER INTERFACE 36
12.1 Introduction 36
12.2 Home Page 36
12.3 Prediction Dashboard 36
12.4 Splunk Dashboard 36
12.5 User Interface Advantages 36
Chapter Summary 36
CHAPTER 13 37
RESULTS AND DISCUSSION 37
13.1 Introduction 37

13.2 Model Performance 37
13.3 Prediction Results 37
13.4 Dashboard Results 37
13.5 Splunk Results 37
13.6 Discussion 37
Chapter Summary 37
CHAPTER 14 38
ADVANTAGES 38
CHAPTER 15 39
LIMITATIONS 39
CHAPTER 16 40
FUTURE SCOPE 40
CHAPTER 17 41
CONCLUSION 41
REFERENCES 42
APPENDIX 43

CHAPTER 1
INTRODUCTION

1.1 Introduction

With the rapid advancement of digital technologies, organizations increasingly rely on computer networks for communication, financial transactions, cloud computing, healthcare, education, and critical infrastructure. As dependency on networked systems grows, cyber threats such as malware, denial-of-service attacks, phishing, ransomware, and unauthorized access attempts have become more frequent and sophisticated. These attacks can lead to severe financial losses, operational disruption, and compromise of sensitive information.

To protect networks against such threats, organizations deploy Intrusion Detection Systems (IDS). An IDS continuously monitors network traffic and system activities to identify suspicious behavior that may indicate a cyber attack. Traditional IDS solutions mainly rely on predefined attack signatures. While effective in detecting known attacks, such systems struggle to identify new or previously unseen attack patterns, and maintaining signature databases requires constant updates, making them less effective against modern cyber threats.

Machine Learning (ML) has emerged as a powerful solution for enhancing intrusion detection by enabling systems to learn from historical network traffic and automatically identify abnormal behavior. Instead of relying solely on signatures, ML models analyze traffic characteristics and classify activities as either normal or malicious based on learned patterns, significantly improving the ability to detect zero-day attacks and evolving cyber threats.

This project presents an AI Intrusion Detection System using Machine Learning and Splunk Enterprise. The system utilizes the Random Forest Classifier, a robust ensemble learning algorithm, to classify network traffic into Normal or Attack categories. The machine learning model is trained using the NSL-KDD dataset, a widely recognized benchmark dataset for intrusion detection research.

To provide an intuitive user experience, a Flask-based web application has been developed that allows users to upload network traffic datasets in CSV format, perform intrusion detection, and visualize prediction results through an interactive dashboard. The dashboard displays attack statistics, prediction confidence scores, graphical charts, and downloadable prediction reports.

Furthermore, the system integrates with Splunk Enterprise, a leading Security Information and Event Management (SIEM) platform [6]. Prediction results are automatically converted into security logs and indexed into Splunk, enabling security analysts to monitor attack trends, visualize intrusion events, and perform real-time security analysis using interactive dashboards.

The proposed solution combines artificial intelligence, web technologies, and SIEM capabilities into a unified platform that improves network security monitoring while providing an educational demonstration of modern cybersecurity techniques.

1.2 Background

Cybersecurity has become one of the most significant concerns in today's digital world. Organizations generate enormous volumes of network traffic every second, making manual monitoring impossible.

Attackers continuously develop sophisticated techniques to bypass conventional security mechanisms. Traditional intrusion detection systems are primarily classified into two categories.

Signature-Based Detection

Signature-based IDS identifies attacks by comparing incoming network traffic with a database of known attack signatures. Although highly accurate for previously identified threats, this approach cannot detect unknown attacks and requires frequent signature updates.

Anomaly-Based Detection

Anomaly-based IDS creates a model of normal network behavior and detects deviations from that behavior. Machine learning significantly enhances anomaly detection by enabling the system to learn complex traffic patterns and classify new observations automatically.

The integration of machine learning into intrusion detection systems has dramatically improved their ability to identify novel attacks while reducing manual intervention.

1.3 Intrusion Detection System (IDS)

An Intrusion Detection System (IDS) is a security solution designed to monitor network traffic or system activities for suspicious behavior. Its primary objective is to detect unauthorized access, malicious activities, policy violations, and potential cyber attacks. The major functions of an IDS include:

- Monitoring network traffic
- Detecting malicious activities
- Generating security alerts
- Logging security events
- Assisting security analysts during incident response

IDS solutions are commonly deployed in enterprise environments, data centers, cloud infrastructures, and government organizations to strengthen cybersecurity defenses.

1.4 Machine Learning in Intrusion Detection

Machine Learning enables intrusion detection systems to automatically identify malicious activities by learning from historical datasets. Instead of relying on manually created rules, ML algorithms recognize patterns within network traffic and classify future traffic based on statistical relationships. The advantages of machine learning in intrusion detection include:

- Detection of previously unseen attacks
- Higher detection accuracy
- Reduced dependence on signature databases
- Automated traffic classification
- Scalability for large datasets
- Continuous learning capability

Among various machine learning algorithms, the Random Forest Classifier [1] provides excellent performance for cybersecurity datasets due to its robustness, high accuracy, and resistance to overfitting.

1.5 Splunk Enterprise Integration

Splunk Enterprise is one of the world's most widely used Security Information and Event Management (SIEM) platforms [6]. It enables organizations to collect, index, search, and visualize machine-generated data in real time. In this project, Splunk Enterprise enhances the AI Intrusion Detection System by:

- Collecting prediction logs
- Displaying attack statistics
- Monitoring intrusion events
- Visualizing attack trends
- Supporting security analysis

This integration demonstrates how machine learning models can work alongside enterprise SIEM solutions to improve security operations.

1.6 Motivation

Traditional intrusion detection systems face limitations in identifying sophisticated cyber attacks. The increasing volume of network traffic makes manual monitoring impractical. Organizations require intelligent systems capable of automatically detecting suspicious activities while providing meaningful visualizations for security analysts. The motivation behind this project is to develop an AI-powered intrusion detection solution that combines Machine Learning, Flask Web Development, Interactive Dashboards, and Splunk Enterprise SIEM into a single platform capable of improving intrusion detection efficiency while serving as an educational cybersecurity project.

1.7 Scope of the Project

The scope of this project includes:

- Development of a Machine Learning-based Intrusion Detection System
- Training a Random Forest classifier using the NSL-KDD dataset
- Designing a Flask-based web application
- Uploading and analyzing network traffic datasets
- Displaying prediction results through interactive dashboards
- Generating downloadable prediction reports
- Creating security log files
- Integrating prediction logs with Splunk Enterprise
- Visualizing security events using SIEM dashboards

The project focuses on detecting binary classes (Normal and Attack) using offline network datasets.

Chapter Summary

This chapter introduced the importance of cybersecurity, intrusion detection systems, and machine learning in network security. It also discussed the motivation, objectives, and overall scope of the proposed AI Intrusion Detection System. The following chapter presents the problem statement addressed by the project.

CHAPTER 2
PROBLEM STATEMENT

2.1 Introduction

The rapid growth of internet usage, cloud computing, and digital transformation has significantly increased the volume of network traffic generated every day. Organizations rely heavily on computer networks for communication, online banking, e-commerce, healthcare, education, government services, and industrial automation. As network usage increases, cyber attackers continuously develop sophisticated techniques to exploit vulnerabilities in network infrastructures.

Traditional security mechanisms such as firewalls primarily focus on preventing unauthorized access based on predefined rules. However, modern cyber attacks frequently bypass these defenses using advanced attack techniques. As a result, organizations require intelligent monitoring systems capable of identifying suspicious activities in real time.

Intrusion Detection Systems (IDS) play a critical role in monitoring network traffic and detecting malicious behavior. However, conventional IDS solutions have several limitations that reduce their effectiveness against modern cyber threats.

2.2 Existing System

Most traditional intrusion detection systems are based on signature matching techniques. These systems compare network traffic against a database of known attack signatures, and whenever a traffic pattern matches a stored signature, an alert is generated. Although signature-based IDS solutions are highly accurate in detecting previously known attacks, they suffer from several disadvantages.

Characteristics of Existing Systems

- Signature-based detection
- Rule-based monitoring
- Manual signature updates
- Limited adaptability
- Static attack database

Examples include Snort IDS, Suricata IDS, and traditional rule-based detection systems. These systems perform well for known attacks but fail to recognize new attack patterns.

2.3 Problems in Existing Systems

Traditional intrusion detection systems face several practical challenges.

1. Inability to Detect Unknown Attacks

Signature-based systems can only detect attacks that already exist in their signature database. If attackers introduce new attack techniques or modify existing malware, the IDS may fail to recognize them.

2. Frequent Signature Updates

Security administrators must continuously update attack signatures. Maintaining these databases is time-consuming and increases operational costs.

3. High False Positive Rate

Many IDS solutions incorrectly classify normal network traffic as malicious, generating unnecessary alerts that increase the workload of security analysts.

4. High False Negative Rate

Some attacks successfully bypass traditional IDS solutions without being detected. Such undetected attacks may result in serious security breaches.

5. Manual Analysis

Large organizations generate millions of network events every day. Manually analyzing these events is difficult, expensive, and time-consuming.

6. Lack of Intelligent Decision Making

Traditional IDS solutions cannot learn from historical attack patterns; they rely entirely on predefined rules rather than adapting to evolving cyber threats.

2.4 Need for the Proposed System

Modern cybersecurity environments require intelligent systems capable of automatically identifying suspicious activities without relying solely on predefined attack signatures. Machine learning algorithms analyze historical network traffic and learn patterns associated with both normal and malicious behavior. This enables the detection of unknown attacks, zero-day attacks, variations of existing attacks, and abnormal network behavior. An AI-powered intrusion detection system significantly improves detection accuracy while reducing manual intervention.

2.5 Proposed System

The proposed system introduces an AI-based Network Intrusion Detection System that combines Machine Learning, Flask, and Splunk Enterprise into a unified cybersecurity solution. The system allows users to upload network traffic datasets in CSV format through a web interface. The uploaded data is preprocessed before being passed to a trained Random Forest classifier, which predicts whether each network connection represents Normal Traffic or Attack Traffic.

Prediction results are displayed through an interactive dashboard that includes total records, attack count, normal count, confidence scores, a prediction table, charts, and downloadable reports. Additionally, prediction results are automatically converted into log files that are indexed into Splunk Enterprise for SIEM visualization.

2.6 Proposed System Architecture Overview

The overall workflow of the proposed system consists of the following stages:

1. User uploads CSV dataset
2. Flask application receives the dataset
3. Data preprocessing begins
4. Random Forest model performs prediction
5. Confidence scores are calculated

6. Dashboard displays results
7. Prediction report is generated
8. Security logs are created
9. Splunk Enterprise visualizes security events

This architecture provides a complete pipeline from raw network traffic to intelligent security monitoring.

2.7 Advantages of the Proposed System

Intelligent Detection

The Random Forest classifier learns from historical data and can identify malicious traffic more effectively.

Higher Accuracy

Machine learning significantly improves prediction accuracy compared to traditional rule-based systems.

User-Friendly Interface

The Flask web application enables users to upload datasets and analyze results without requiring command-line operations.

Interactive Visualization

Charts and graphical summaries improve the understanding of prediction results.

SIEM Integration

Splunk Enterprise provides enterprise-grade visualization and monitoring capabilities.

Automated Reporting

Prediction results are automatically exported into downloadable CSV reports.

Security Log Generation

The system automatically generates security logs suitable for SIEM analysis.

2.8 Applications

The proposed system can be deployed in various environments, including:

- Banking networks
- Healthcare organizations
- Educational institutions
- Government departments
- Cloud infrastructure
- Enterprise data centers
- Security Operation Centers (SOC)
- Research laboratories

Chapter Summary

This chapter discussed the limitations of traditional intrusion detection systems and highlighted the need for intelligent machine learning-based security solutions. The proposed AI Intrusion Detection System addresses these limitations by combining Random Forest classification, Flask web development, and Splunk Enterprise SIEM to improve intrusion detection accuracy and security monitoring.

CHAPTER 3
OBJECTIVES

3.1 Introduction

The primary objective of this project is to develop an intelligent Network Intrusion Detection System capable of accurately detecting malicious network activities using Machine Learning techniques. The proposed system aims to overcome the limitations of conventional signature-based intrusion detection systems by employing a Random Forest classifier trained on the NSL-KDD dataset [2], [10]. Furthermore, the project integrates Splunk Enterprise to provide Security Information and Event Management (SIEM) capabilities for centralized monitoring and visualization of intrusion events.

3.2 General Objective

The general objective of this project is to design and implement an AI-powered Intrusion Detection System that automatically analyzes network traffic, predicts malicious activities using Machine Learning, and visualizes security events through a professional web interface and Splunk Enterprise dashboard.

3.3 Specific Objectives

Objective 1 – Develop an AI-Based Intrusion Detection Model

Develop a machine learning model capable of distinguishing between normal and malicious network traffic using the Random Forest algorithm.

Objective 2 – Utilize the NSL-KDD Dataset

Train and evaluate the machine learning model using the NSL-KDD dataset, which is a widely accepted benchmark dataset for intrusion detection research.

Objective 3 – Perform Data Preprocessing

Prepare the dataset by handling categorical attributes, encoding labels, and formatting the data into a machine learning-friendly structure.

Objective 4 – Develop a Flask Web Application

Create a responsive web application that enables users to upload network traffic datasets and receive prediction results through an intuitive interface.

Objective 5 – Generate Prediction Reports

Automatically generate downloadable CSV reports containing prediction results for further analysis and documentation.

Objective 6 – Display Interactive Visualizations

Present prediction statistics using interactive charts and summary cards that enhance user understanding of intrusion detection results.

Objective 7 – Generate Security Logs

Automatically create structured log files that record prediction results and security events for further monitoring.

Objective 8 – Integrate Splunk Enterprise

Import generated log files into Splunk Enterprise and visualize attack trends using a SIEM dashboard.

Objective 9 – Improve Detection Accuracy

Achieve high prediction accuracy while minimizing false positive and false negative rates using the Random Forest classifier.

Objective 10 – Build a Scalable Security Solution

Design the application in a modular architecture that allows future integration with live packet capture, cloud deployment, REST APIs, and advanced machine learning algorithms.

3.4 Expected Outcomes

Upon successful completion of the project, the following outcomes are expected:

- Accurate prediction of network intrusions
- Automated classification of network traffic
- Interactive visualization of prediction statistics
- Downloadable prediction reports
- Automatic security log generation
- Splunk Enterprise dashboard for monitoring
- Improved understanding of AI-based intrusion detection systems

Chapter Summary

This chapter presented the objectives of the proposed AI Intrusion Detection System. The objectives focus on building an intelligent intrusion detection model, developing a user-friendly web application, generating security logs, and integrating SIEM technology for effective network monitoring.

CHAPTER 4
LITERATURE SURVEY

4.1 Introduction

The rapid increase in cyber attacks has encouraged researchers to develop advanced intrusion detection techniques capable of identifying malicious activities more efficiently. Over the past two decades, intrusion detection has evolved from simple rule-based systems to intelligent machine learning models capable of detecting complex attack patterns. This chapter reviews existing research related to intrusion detection systems, machine learning techniques, Random Forest classifiers, the NSL-KDD dataset, and Security Information and Event Management (SIEM) platforms.

4.2 Intrusion Detection Systems

An Intrusion Detection System (IDS) continuously monitors network traffic or host activities to detect unauthorized access and malicious behavior. Its primary objective is to identify suspicious activities before they cause significant damage to organizational resources. IDS solutions are broadly classified into two categories.

Host-Based Intrusion Detection System (HIDS)

A Host-Based IDS monitors activities occurring within a single computer system, such as file integrity monitoring, system log analysis, registry monitoring, and user activity monitoring.

Advantages: detailed system-level monitoring, detects insider threats, and monitors file modifications.

Disadvantages: limited to individual hosts and higher resource consumption.

Network-Based Intrusion Detection System (NIDS)

A Network-Based IDS monitors packets traveling across a network.

Advantages: centralized monitoring, detects attacks across multiple systems, and is suitable for enterprise environments.

Disadvantages: high-speed traffic processing requirements and difficulty detecting encrypted traffic.

The proposed project belongs to the Network-Based Intrusion Detection System (NIDS) category.

4.3 Signature-Based Detection

Signature-based IDS compares incoming traffic with a predefined database of attack signatures. Examples include Snort and Suricata.

Advantages: high accuracy for known attacks and low false positives.

Limitations: cannot detect zero-day attacks and requires continuous signature updates.

4.4 Anomaly-Based Detection

Anomaly detection identifies deviations from normal network behavior. Machine learning algorithms are commonly used for anomaly detection because they learn traffic characteristics from historical data.

Advantages: detects unknown attacks, adaptive learning, and suitability for evolving cyber threats.

Disadvantages: may generate higher false positives and requires quality training data.

4.5 Machine Learning in Intrusion Detection

Machine Learning enables computers to learn patterns from historical network traffic without explicitly programmed rules. Common machine learning algorithms used in intrusion detection include:

- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- Naïve Bayes
- K-Nearest Neighbors (KNN)
- Logistic Regression
- Artificial Neural Networks

Among these, Random Forest has consistently demonstrated excellent performance due to its high accuracy and robustness [1].

4.6 Random Forest Classifier

Random Forest is an ensemble learning algorithm introduced by Leo Breiman [1]. It constructs multiple decision trees and combines their outputs using majority voting.

Advantages include high prediction accuracy, reduced overfitting, the ability to handle high-dimensional data, robustness against noisy datasets, and suitability for cybersecurity applications. Because of these advantages, Random Forest was selected as the classification algorithm for this project.

4.7 NSL-KDD Dataset

The NSL-KDD dataset is an improved version of the KDD Cup 1999 dataset and is widely used for evaluating intrusion detection systems [2], [10]. Advantages of NSL-KDD include removal of duplicate records, balanced training and testing sets, a standard benchmark for IDS research, and improved evaluation reliability. The dataset contains network traffic labeled as either Normal or various categories of attacks.

4.8 Security Information and Event Management (SIEM)

SIEM platforms collect and analyze security logs from multiple sources. Major SIEM platforms include Splunk Enterprise [6], IBM QRadar, Microsoft Sentinel, ArcSight, and Elastic SIEM. Splunk Enterprise was selected because of its powerful search capabilities, dashboard creation, real-time monitoring, and ease of integration with generated log files.

4.9 Research Gap

Although numerous intrusion detection systems have been proposed, many existing solutions focus only on prediction accuracy and lack integrated visualization or practical deployment. The identified research gaps include:

- Limited user-friendly interfaces

- Lack of downloadable prediction reports
- Absence of SIEM integration
- Limited visualization of intrusion events
- Minimal educational demonstrations

The proposed project addresses these gaps by combining Machine Learning, Flask, and Splunk Enterprise into a unified solution.

Chapter Summary

The literature survey indicates that machine learning-based intrusion detection significantly outperforms traditional signature-based systems in detecting unknown attacks. Random Forest provides excellent prediction performance, while Splunk Enterprise enhances monitoring through SIEM dashboards. Integrating these technologies results in a practical, scalable, and educational intrusion detection platform.

CHAPTER 5
SYSTEM REQUIREMENTS

5.1 Introduction

System requirements define the minimum hardware and software resources required to successfully develop, execute, and maintain the proposed AI Intrusion Detection System. Proper system specifications ensure smooth execution of machine learning algorithms, web application deployment, and Splunk Enterprise integration. The project has been developed using open-source technologies, making it cost-effective and easily deployable on standard desktop or laptop computers.

5.2 Hardware Requirements

Table I lists the minimum hardware configuration required for successful execution of the proposed system.

Component	Specification
Processor	Intel Core i5 / Intel Core i7 or AMD Ryzen 5
RAM	Minimum 8 GB
Storage	256 GB SSD or higher
Display	1366 × 768 resolution or higher
Network	Internet connection
Keyboard & Mouse	Standard input devices

TABLE I. HARDWARE REQUIREMENTS

Processor

The Random Forest algorithm performs numerous computations while training and predicting network traffic. Therefore, a multi-core processor is recommended.

Memory

Machine learning libraries such as Pandas and Scikit-learn require sufficient memory for loading and processing the NSL-KDD dataset.

Storage

Solid State Drives (SSD) significantly improve dataset loading time, model execution speed, and application responsiveness.

5.3 Software Requirements

Table II lists the software tools used during development of the project.

Software	Version
Operating System	Windows 11
Python	3.10 or higher

Software	Version
Flask	Latest stable version
Visual Studio Code	Latest version
Git	Latest version
GitHub	Cloud repository
Splunk Enterprise	Latest version
Google Chrome / Microsoft Edge	Latest version

TABLE II. SOFTWARE REQUIREMENTS

5.4 Python Libraries

Table III summarizes the Python libraries used throughout the project.

Library	Purpose
Pandas	Data manipulation
NumPy	Numerical computation
Scikit-learn	Machine learning
Flask	Web framework
Pickle	Model serialization
Joblib	Saving machine learning models
Matplotlib	Data visualization
Chart.js	Dashboard charts

TABLE III. PYTHON LIBRARIES

5.5 Development Environment

The complete project was developed using the following environment:

- Operating System: Windows 11
- IDE: Visual Studio Code
- Language: Python
- Version Control: Git
- Repository Hosting: GitHub
- SIEM Platform: Splunk Enterprise
- Browser: Microsoft Edge

Chapter Summary

This chapter discussed the hardware and software requirements necessary for developing and executing the AI Intrusion Detection System. The next chapter explains the technologies used during implementation.

CHAPTER 6
TECHNOLOGIES USED

6.1 Introduction

The AI Intrusion Detection System combines multiple technologies from machine learning, web development, cybersecurity, and data visualization to create a complete intelligent intrusion detection platform. Each technology contributes a specific role in the overall architecture.

6.2 Python

Python is a high-level programming language widely used in Artificial Intelligence, Data Science, Machine Learning, and Cybersecurity [5]. Reasons for choosing Python include simple syntax, extensive library support, fast development, cross-platform compatibility, and an excellent machine learning ecosystem. Python serves as the core programming language of the entire project.

6.3 Flask

Flask is a lightweight Python web framework [4]. In this project, Flask provides file upload functionality, prediction request handling, dashboard generation, HTML rendering, and downloadable reports, enabling seamless interaction between users and the machine learning model.

6.4 Pandas

Pandas is used for reading, preprocessing, and manipulating network traffic datasets. Major tasks include reading CSV files, cleaning data, selecting features, and exporting prediction results.

6.5 NumPy

NumPy provides efficient numerical computation. Its responsibilities include matrix operations, numerical calculations, feature transformation, and data conversion.

6.6 Scikit-learn

Scikit-learn is one of the most popular Machine Learning libraries [3]. It provides the Random Forest Classifier, model evaluation, prediction functions, and performance metrics.

6.7 Bootstrap

Bootstrap is used to design a professional and responsive web interface. Advantages include mobile responsiveness, professional appearance, predefined UI components, and easy customization.

6.8 HTML

HTML structures the web pages and is responsible for upload forms, tables, buttons, and navigation.

6.9 CSS

CSS improves the visual appearance of the web application, controlling colors, fonts, layout, animations, and responsive design.

6.10 JavaScript

JavaScript enhances user interaction, enabling dynamic updates, interactive components, and client-side processing.

6.11 Chart.js

Chart.js is used to generate graphical representations of prediction results, including pie charts, bar charts, attack distribution, and prediction statistics.

6.12 Git & GitHub

Git manages version control. GitHub hosts the project online and provides source code management, collaboration, documentation, and portfolio presentation.

6.13 Splunk Enterprise

Splunk Enterprise is a Security Information and Event Management (SIEM) platform [6]. Its functions include log indexing, security event monitoring, dashboard creation, alert visualization, and Search Processing Language (SPL). The generated prediction logs are imported into Splunk to create an enterprise-level monitoring dashboard.

Chapter Summary

This chapter explained the technologies used throughout the project. Together, these technologies provide machine learning capabilities, web application functionality, cybersecurity monitoring, and interactive visualization.

CHAPTER 7
DATASET

7.1 Introduction

The performance of any machine learning model depends heavily on the quality of the training dataset. The proposed AI Intrusion Detection System uses the NSL-KDD dataset [2], [10], one of the most widely accepted benchmark datasets for evaluating intrusion detection systems. The dataset contains labeled network connection records representing both normal and malicious network activities.

7.2 NSL-KDD Dataset

The NSL-KDD dataset was developed as an improved version of the KDD Cup 1999 dataset. The original KDD dataset contained a large number of duplicate records, resulting in biased machine learning models. The NSL-KDD dataset removes these duplicate entries and provides balanced training and testing datasets.

7.3 Dataset Files

The project uses two primary files, summarized in Table IV.

File	Purpose
KDDTrain+	Training dataset
KDDTest+	Testing dataset

TABLE IV. DATASET FILES

These datasets contain labeled network traffic records used for model training and evaluation.

7.4 Dataset Features

Each network connection contains 41 input features describing different characteristics of network traffic. Examples include:

- Duration
- Protocol type
- Service
- Source bytes
- Destination bytes
- Number of failed logins
- Logged-in status
- Number of file creations
- Number of shell prompts
- Connection count
- Service count

The target attribute identifies whether the connection is Normal or an Attack.

7.5 Data Preprocessing

Raw datasets cannot be directly used by machine learning algorithms. Several preprocessing operations are performed.

Data Cleaning

Invalid or inconsistent records are removed.

Feature Selection

Only relevant features are selected for training.

Label Encoding

Categorical variables such as protocol type and service are converted into numerical values.

Data Transformation

The processed data is transformed into a format suitable for the Random Forest classifier.

7.6 Dataset Workflow

10. Load NSL-KDD dataset
11. Read CSV file
12. Preprocess features
13. Encode labels
14. Train Random Forest model
15. Save trained model
16. Predict new data

7.7 Advantages of NSL-KDD

- Benchmark dataset for IDS research
- Reduced redundancy
- Balanced data distribution
- Reliable evaluation
- Widely accepted by researchers

7.8 Limitations of the Dataset

Despite its popularity, the dataset has some limitations:

- Offline dataset
- Older attack patterns
- Does not represent current real-time internet traffic
- Limited attack diversity compared to modern datasets

Future work may utilize datasets such as CICIDS2017, UNSW-NB15, or TON_IoT for more contemporary evaluation.

Chapter Summary

This chapter discussed the NSL-KDD dataset, its structure, preprocessing techniques, advantages, and limitations. The next chapter explains the system architecture and describes how different components of the AI Intrusion Detection System interact to perform intrusion detection.

CHAPTER 8
SYSTEM ARCHITECTURE

8.1 Introduction

System architecture defines the structural design of the proposed AI Intrusion Detection System and illustrates how different software components interact to perform intrusion detection. The architecture integrates machine learning, web technologies, data preprocessing, reporting, and Security Information and Event Management (SIEM) into a unified framework.

The proposed architecture follows a modular design, allowing each component to perform a specific function independently while communicating efficiently with other modules. This modularity improves scalability, maintainability, and future extensibility.

8.2 System Architecture Diagram

[Insert architecture.png here]

Figure 1: Architecture of the AI Intrusion Detection System

8.3 Architecture Components

8.3.1 User Interface

The user interacts with the system through a Flask-based web application responsible for uploading network traffic datasets, viewing prediction results, downloading reports, and monitoring statistics. The web interface has been designed using HTML, CSS, Bootstrap, and JavaScript to provide a responsive and user-friendly experience.

8.3.2 CSV Upload Module

This module accepts CSV files containing network traffic records and performs file validation, upload handling, and storage in the uploads directory. Only properly formatted datasets are accepted for further processing.

8.3.3 Data Preprocessing Module

Before prediction, uploaded data undergoes preprocessing that performs missing value handling, feature selection, label encoding, data transformation, and formatting, ensuring the dataset matches the format used during model training.

8.3.4 Machine Learning Prediction Module

The preprocessed dataset is passed to the trained Random Forest classifier, which performs feature evaluation, classification, and confidence score calculation. Each network record is classified as either Normal or Attack.

8.3.5 Prediction Dashboard

Prediction results are displayed through an interactive dashboard that includes total records, normal traffic, attack traffic, confidence scores, a prediction table, a pie chart, and a bar chart, enabling users to quickly understand the security status of the uploaded dataset.

8.3.6 Report Generation Module

After prediction, the application automatically generates a prediction CSV, confidence scores, and a downloadable report that can be used for further analysis and documentation.

8.3.7 Security Log Generation

The application automatically generates structured security logs, each containing a timestamp, prediction, confidence score, and status. These logs serve as the input for Splunk Enterprise.

8.3.8 Splunk Enterprise

Splunk Enterprise indexes the generated security logs and provides Search Processing Language (SPL), dashboards, charts, alert monitoring, and security analytics, transforming prediction results into enterprise-level security monitoring.

8.4 Overall Workflow

17. User uploads CSV dataset
18. Flask receives dataset
19. Data preprocessing begins
20. Random Forest predicts traffic
21. Confidence scores generated
22. Dashboard displays results
23. CSV report generated
24. Security logs created
25. Splunk indexes logs
26. SIEM dashboard visualizes alerts

8.5 Advantages of Modular Architecture

- Easy maintenance
- High scalability
- Independent modules
- Faster debugging
- Better code organization
- Future enhancements

Chapter Summary

This chapter described the architecture of the proposed AI Intrusion Detection System. Each module contributes toward efficient intrusion detection, visualization, reporting, and SIEM integration.

CHAPTER 9
METHODOLOGY

9.1 Introduction

Methodology describes the systematic procedure followed during the development and execution of the AI Intrusion Detection System. It explains how network traffic is processed from upload to prediction and visualization. The methodology consists of nine sequential phases.

9.2 Phase 1 – Dataset Collection

The NSL-KDD dataset is collected for model training and testing using the KDDTrain+ and KDDTest+ files, which contain labeled network traffic representing normal and attack connections.

9.3 Phase 2 – Data Preprocessing

The collected dataset undergoes preprocessing, including cleaning, encoding categorical features, feature selection, and formatting to improve prediction accuracy.

9.4 Phase 3 – Model Training

The Random Forest algorithm is trained using the processed dataset. Training involves feature learning, decision tree generation, and ensemble construction. The trained model is stored using Pickle.

9.5 Phase 4 – Web Application Development

A Flask application is developed to provide a graphical user interface through which users can upload CSV files, view predictions, and download reports.

9.6 Phase 5 – Prediction

Uploaded datasets are passed to the trained model, which predicts Normal or Attack labels along with confidence scores.

9.7 Phase 6 – Dashboard Generation

Prediction statistics are displayed through summary cards, a pie chart, a bar chart, and a prediction table, simplifying security analysis.

9.8 Phase 7 – Report Generation

Prediction results are exported as predictions.csv and other downloadable reports.

9.9 Phase 8 – Log Generation

The application generates security logs, each containing a timestamp, prediction, confidence, and status.

9.10 Phase 9 – Splunk Integration

Generated logs are imported into Splunk Enterprise. Splunk dashboards display attack count, normal count, timeline, and security alerts, enabling centralized monitoring.

9.11 Methodology Flow

The methodology follows the sequence: Dataset → Preprocessing → Model Training → Flask Application → Prediction → Dashboard → Reports → Logs → Splunk Dashboard.

Chapter Summary

This chapter explained the step-by-step methodology used to develop the proposed intrusion detection system.

CHAPTER 10

RANDOM FOREST ALGORITHM

10.1 Introduction

Random Forest is an ensemble machine learning algorithm widely used for classification and regression problems. It combines multiple decision trees to improve prediction accuracy and reduce overfitting. The algorithm was introduced by Leo Breiman in 2001 [1] and is recognized as one of the most effective supervised learning algorithms.

10.2 Working Principle

Instead of building a single decision tree, Random Forest constructs multiple trees. Each tree receives random samples, uses random feature subsets, and predicts independently. The final prediction is determined using majority voting.

10.3 Random Forest Workflow

27. Select random samples
28. Construct multiple decision trees
29. Predict class for each tree
30. Count votes
31. Choose the majority class

This ensemble approach improves robustness and generalization.

10.4 Why Random Forest?

Random Forest was selected because of its high accuracy, fast prediction, resistance to overfitting, ability to handle large datasets, ability to handle missing values, and excellent performance on tabular data. These characteristics make it suitable for intrusion detection tasks.

10.5 Advantages

- High prediction accuracy
- Robust against noisy data
- Less overfitting
- Parallel processing
- Suitable for large datasets
- Feature importance estimation

10.6 Disadvantages

- Larger model size
- Higher memory usage
- More complex than a single decision tree
- Longer training time for very large datasets

10.7 Why It Was Used in This Project

The AI Intrusion Detection System requires a classifier capable of accurately distinguishing between normal and malicious network traffic. Random Forest provides excellent classification performance, stable predictions, low false positives, and high generalization capability. Therefore, it was selected as the core machine learning algorithm.

10.8 Performance

The trained Random Forest model achieved excellent performance on the NSL-KDD dataset, as summarized in Table V.

Metric	Value
Accuracy	99.96%
Precision	1.00
Recall	1.00
F1-Score	1.00

TABLE V. MODEL PERFORMANCE METRICS

[Insert confusion matrix / classification report figure here, if available]

Chapter Summary

This chapter explained the Random Forest algorithm, its working principle, advantages, disadvantages, and the reasons for selecting it for the proposed AI Intrusion Detection System. The algorithm's ensemble learning approach provides high accuracy and reliability for classifying network traffic.

CHAPTER 11

SYSTEM IMPLEMENTATION

11.1 Introduction

System implementation is the practical realization of the proposed AI Intrusion Detection System. This phase integrates the machine learning model, web application, visualization components, and Splunk Enterprise into a fully functional intrusion detection platform. The implementation follows a modular architecture where each component performs a specific task independently, improving maintainability, scalability, and future development.

11.2 Project Directory Structure

The project follows the directory structure shown below.

```
AI-Intrusion-Detection/ ├── app.py ├── requirements.txt ├── README.md ├── dataset/ │   ├── KDDTrain+.txt │   ├── KDDTest+.txt │   ├── processed_train.csv │   ├── processed_test.csv ├── models/ │   ├── model.pkl │   └── label_encoders.pkl ├── src/ │   ├── train_model.py │   ├── preprocess.py │   ├── predict.py │   ├── config.py │   └── utils.py ├── templates/ │   ├── index.html │   └── results.html ├── static/ │   ├── css/ │   └── js/ ├── uploads/ ├── outputs/ ├── logs/ └── screenshots/
```

The modular directory structure simplifies maintenance and future enhancements.

11.3 Module Description

11.3.1 Dataset Module

This module stores the NSL-KDD training and testing datasets used for model development and evaluation, including the training dataset, testing dataset, and processed datasets.

11.3.2 Model Module

The trained Random Forest model is stored in serialized format using Pickle, as model.pkl and label_encoders.pkl. These files eliminate the need to retrain the model whenever the application starts.

11.3.3 Preprocessing Module

This module prepares uploaded datasets before prediction, handling feature encoding, data formatting, feature validation, and dataset transformation.

11.3.4 Prediction Module

The prediction module loads the trained Random Forest model and performs intrusion detection, producing outputs of Normal, Attack, and a confidence score.

11.3.5 Flask Application

The Flask application serves as the central controller, responsible for uploading datasets, calling preprocessing functions, performing predictions, displaying dashboards, and exporting reports.

11.3.6 Dashboard Module

The dashboard presents prediction results using interactive cards and graphical visualizations, including total records, normal traffic, attack traffic, confidence scores, a pie chart, and a bar chart.

11.3.7 Log Generation Module

Prediction results are automatically converted into structured log files, which are later imported into Splunk Enterprise.

11.3.8 Splunk Integration Module

Generated logs are indexed into Splunk. Splunk dashboards display attack count, normal count, security timeline, and event statistics.

11.4 Execution Flow

32. Application starts
33. User uploads CSV dataset
34. Dataset is preprocessed
35. Random Forest predicts labels
36. Dashboard displays results
37. Prediction report generated
38. Security logs created
39. Splunk visualizes events

Chapter Summary

This chapter explained how different software modules work together to implement the proposed AI Intrusion Detection System.

CHAPTER 12
USER INTERFACE

12.1 Introduction

A user-friendly interface improves accessibility and allows users without machine learning expertise to perform intrusion detection easily. The application provides a clean and responsive web interface developed using Flask, Bootstrap, HTML, CSS, and JavaScript.

12.2 Home Page

[Insert screenshot: home_page.png]

Figure 2: Home Page

The home page allows users to upload CSV datasets, start intrusion detection, and navigate through the application. Features include a responsive design, professional appearance, and simple navigation.

12.3 Prediction Dashboard

[Insert screenshot: dashboard.png]

Figure 3: Prediction Dashboard

The dashboard displays total records, attack count, normal count, a prediction table, confidence scores, a pie chart, and a bar chart, providing an overview of prediction results and improving understanding through graphical representation.

12.4 Splunk Dashboard

[Insert screenshot: splunk_dashboard.png]

Figure 4: Splunk Dashboard

Splunk Enterprise visualizes attack trends, security events, indexed logs, and event distribution, demonstrating SIEM functionality and supporting security monitoring.

12.5 User Interface Advantages

- Simple navigation
- Modern responsive design
- Easy dataset upload
- Clear visualization
- Downloadable reports
- Professional appearance

Chapter Summary

The user interface combines usability with functionality, enabling efficient interaction with the intrusion detection system.

CHAPTER 13
RESULTS AND DISCUSSION

13.1 Introduction

The proposed AI Intrusion Detection System was successfully developed and tested using the NSL-KDD dataset. The system accurately classifies network traffic and presents results through an interactive dashboard.

13.2 Model Performance

The Random Forest classifier achieved excellent classification performance, as summarized in Table VI.

Metric	Value
Accuracy	99.96%
Precision	1.00
Recall	1.00
F1-Score	1.00

TABLE VI. FINAL MODEL PERFORMANCE (REPLACE WITH ACTUAL RESULTS IF DIFFERENT)

13.3 Prediction Results

The application successfully classified uploaded network traffic into Normal and Attack categories. Each prediction includes a confidence score, helping users evaluate the certainty of the model's decision.

13.4 Dashboard Results

The prediction dashboard displays summary cards, a prediction table, attack distribution, confidence levels, and downloadable CSV reports, simplifying analysis and improving user understanding.

13.5 Splunk Results

Generated security logs were indexed successfully into Splunk Enterprise. The Splunk dashboard displays attack frequency, normal traffic, event timeline, and log statistics, demonstrating the successful integration of AI-based prediction with SIEM monitoring.

13.6 Discussion

The project successfully combines machine learning and SIEM technologies into a practical intrusion detection platform. The system demonstrates high prediction accuracy, fast execution, a professional web interface, effective visualization, and enterprise-style security monitoring.

Chapter Summary

The experimental results demonstrate that the proposed AI Intrusion Detection System performs efficiently and provides meaningful security insights.

CHAPTER 14
ADVANTAGES

The proposed system offers several advantages:

- High intrusion detection accuracy
- Automated attack classification
- User-friendly web application
- Professional dashboard
- Confidence score generation
- Downloadable reports
- Security log generation
- Splunk Enterprise integration
- Modular architecture
- Easy future expansion

CHAPTER 15
LIMITATIONS

Despite its effectiveness, the current implementation has several limitations:

- Offline dataset
- No live packet capture
- Binary classification only
- Single machine learning algorithm
- Local deployment
- Limited real-time capabilities

These limitations provide opportunities for future research and development.

CHAPTER 16
FUTURE SCOPE

Several enhancements can further improve the system. Future developments include:

- Real-time packet capture
- Deep Learning (LSTM, CNN)
- Multi-class attack classification
- Docker containerization
- Cloud deployment (AWS, Azure)
- REST API integration
- Threat intelligence feeds
- Email/SMS alerts
- User authentication
- Role-based access control
- Automatic model retraining
- Real-time SOC alerts

CHAPTER 17
CONCLUSION

The AI Intrusion Detection System developed in this project successfully demonstrates the integration of Machine Learning, Web Development, and SIEM technologies for intelligent network security monitoring.

The Random Forest classifier [1] effectively detects malicious network traffic using the NSL-KDD dataset [2], [10], while the Flask web application [4] provides an intuitive interface for dataset upload, prediction, and visualization.

The integration with Splunk Enterprise [6] extends the system beyond prediction by enabling centralized security monitoring, event visualization, and SOC-style dashboards.

The project highlights the practical application of artificial intelligence in cybersecurity and demonstrates how machine learning can significantly improve intrusion detection capabilities compared to traditional signature-based systems.

The modular architecture, professional user interface, and extensible design make the system suitable for educational purposes, portfolio presentation, and future research.

REFERENCES

[1] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[2] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA), 2009, pp. 1–6.

[3] Scikit-learn Developers, Scikit-learn User Guide. [Online]. Available: https://scikit-learn.org

[4] Flask Documentation, The Flask Web Framework Documentation. [Online]. Available: https://flask.palletsprojects.com

[5] Python Software Foundation, Python Documentation. [Online]. Available: https://docs.python.org

[6] Splunk Inc., Splunk Enterprise Documentation. [Online]. Available: https://docs.splunk.com

[7] W. Stallings, Network Security Essentials: Applications and Standards. Boston, MA, USA: Pearson.

[8] W. Stallings, Cryptography and Network Security: Principles and Practice. Boston, MA, USA: Pearson.

[9] OWASP Foundation, OWASP Top 10 Web Application Security Risks. [Online]. Available: https://owasp.org

[10] NSL-KDD Dataset Documentation, Canadian Institute for Cybersecurity, University of New Brunswick. [Online]. Available: https://www.unb.ca/cic/datasets/nsl.html

APPENDIX

The following supplementary materials are included as part of the complete project documentation:

- GitHub repository link
- Project folder structure
- requirements.txt
- Architecture diagram
- Home page screenshot
- Prediction dashboard screenshot
- Splunk dashboard screenshot
- Sample prediction CSV
- Sample security log
- Installation steps
- User manual